

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 8
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в ширину»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.

Гуляевский Д.С.

Каташов К.А.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Обход графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.
3. *Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2

1. *Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной.
2. *Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс **queue** и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Описание метода решения

Метод решения основан на алгоритме обхода графа в ширину (BFS) с использованием различных комбинаций структур данных.

На первом этапе создается матрица смежности графа. Пользователь вводит количество вершин, после чего программа динамически выделяет память для квадратной матрицы. Матрица заполняется случайными значениями: для каждой пары вершин i и j , кроме случаев когда i равно j , генерируется случайное число 0 или 1, определяющее наличие или отсутствие ребра между этими вершинами. Поскольку граф неориентированный, матрица делается симметричной - значение для пары (i,j) всегда равно значению для пары (j,i) . На главной диагонали

матрицы устанавливаются нули, что исключает наличие петель. После генерации матрица выводится на экран для визуализации структуры графа.

На втором этапе выполняется обход графа в ширину. Пользователь выбирает начальную вершину, с которой начинается обход. Алгоритм использует очередь для хранения вершин, которые нужно обработать. Начальная вершина помечается как посещенная и помещается в очередь. Затем в цикле из очереди извлекается очередная вершина, она выводится на экран как часть порядка обхода, и все смежные с ней непосещенные вершины добавляются в конец очереди и помечаются как посещенные. Этот процесс продолжается до тех пор, пока очередь не станет пустой, что означает, что все достижимые из начальной вершины узлы были обработаны. В результате на экран выводится последовательность вершин в порядке их посещения при обходе в ширину.

Листинг

Задание 1

```
#include <iostream>
#include <locale.h>
#include <time.h>
#include <cstdio>
#include <queue>
using namespace std;

void BFS(int** G, int* vis, int numG, int s) {
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);
        for (int i = 0; i < numG; i++) {
            if (vis[i] == 0 && G[v][i] == 1) {
                q.push(i);
                vis[i] = 1;
            }
        }
    }
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int** G;
    int* vis;
    int numG, s;

    std::cout << "Введите кол-во элементов: " << std::endl;
    std::cin >> numG;

    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    vis = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        vis[i] = 0;
        for (int j = 0; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d ", G[i][j]);
        }
        printf("\n");
    }
    printf("Введите нач вершину: ");
    scanf_s("%d", &s);
    printf("Порядок обхода: ");
    BFS(G, vis, numG, s);
    return 0;
}
```

Задание 2

```

#include <iostream>
#include <vector>
#include <queue>
#include <chrono>
#include <memory>

// Узел для собственной реализации списка
template<typename T>
class ListNode {
public:
    T data;
    std::shared_ptr<ListNode<T>> next;

    ListNode(const T& value) : data(value), next(nullptr) {}
};

// Самостоятельная реализация очереди на основе списка
template<typename T>
class CustomQueue {
private:
    std::shared_ptr<ListNode<T>> front;
    std::shared_ptr<ListNode<T>> rear;
    size_t queueSize;

public:
    CustomQueue() : front(nullptr), rear(nullptr), queueSize(0) {}

    // Добавление элемента в очередь
    void push(const T& value) {
        auto newNode = std::make_shared<ListNode<T>>(value);
        if (rear == nullptr) {
            front = rear = newNode;
        }
        else {
            rear->next = newNode;
            rear = newNode;
        }
        queueSize++;
    }

    // Удаление элемента из очереди
    void pop() {
        if (empty()) {
            throw std::runtime_error("Queue is empty");
        }

        front = front->next;
        if (front == nullptr) {
            rear = nullptr;
        }
        queueSize--;
    }

    // Получение первого элемента
    T& getFront() {
        if (empty()) {
            throw std::runtime_error("Queue is empty");
        }
        return front->data;
    }

    // Проверка на пустоту
    bool empty() const {
        return front == nullptr;
    }

    // Получение размера очереди

```

```

    size_t size() const {
        return queueSize;
    }
};

```

// Класс для представления графа в матричной форме

```

class Graph {

```

```

private:

```

```

    std::vector<std::vector<int>> adjacencyMatrix;

```

```

    int vertices;

```

```

public:

```

```

    Graph(int n) : vertices(n) {

```

```

        adjacencyMatrix.resize(n, std::vector<int>(n, 0));

```

```

    }

```

// Добавление ребра

```

    void addEdge(int u, int v) {

```

```

        if (u >= 0 && u < vertices && v >= 0 && v < vertices) {

```

```

            adjacencyMatrix[u][v] = 1;

```

```

            adjacencyMatrix[v][u] = 1; // Для неориентированного графа

```

```

        }

```

```

    }

```

// Генерация случайного графа

```

    void generateRandomGraph(double density = 0.3) {

```

```

        srand(time(nullptr));

```

```

        for (int i = 0; i < vertices; i++) {

```

```

            for (int j = i + 1; j < vertices; j++) {

```

```

                if (static_cast<double>(rand()) / RAND_MAX < density) {

```

```

                    addEdge(i, j);

```

```

                }

```

```

            }

```

```

        }

```

```

    }

```

// BFS с использованием стандартной очереди

```

    std::vector<int> bfsWithStdQueue(int startVertex) {

```

```

        std::vector<int> traversalOrder;

```

```

        std::vector<bool> visited(vertices, false);

```

```

        std::queue<int> q;

```

```

        visited[startVertex] = true;

```

```

        q.push(startVertex);

```

```

        while (!q.empty()) {

```

```

            int current = q.front();

```

```

            q.pop();

```

```

            traversalOrder.push_back(current);

```

```

            for (int i = 0; i < vertices; i++) {

```

```

                if (adjacencyMatrix[current][i] == 1 && !visited[i]) {

```

```

                    visited[i] = true;

```

```

                    q.push(i);

```

```

                }

```

```

            }

```

```

        }

```

```

        return traversalOrder;

```

```

    }

```

// BFS с использованием собственной очереди

```

    std::vector<int> bfsWithCustomQueue(int startVertex) {

```

```

        std::vector<int> traversalOrder;

```

```

        std::vector<bool> visited(vertices, false);

```

```

        CustomQueue<int> q;

```

```

        visited[startVertex] = true;
        q.push(startVertex);

        while (!q.empty()) {
            int current = q.getFront();
            q.pop();
            traversalOrder.push_back(current);

            for (int i = 0; i < vertices; i++) {
                if (adjacencyMatrix[current][i] == 1 && !visited[i]) {
                    visited[i] = true;
                    q.push(i);
                }
            }
        }

        return traversalOrder;
    }

    // Вывод матрицы смежности (для отладки)
    void printAdjacencyMatrix() {
        std::cout << "Матрица смежности:\n";
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                std::cout << adjacencyMatrix[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    int getVerticesCount() const {
        return vertices;
    }
};

// Функция для измерения времени выполнения
template<typename Func>
double measureTime(Func func) {
    auto start = std::chrono::high_resolution_clock::now();
    func();
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;
    return duration.count();
}

int main() {
    setlocale(LC_ALL, "rus");
    std::vector<int> graphSizes = { 10, 50, 100, 200, 500, 1000 };
    int startVertex = 0;

    std::cout << "Сравнение времени выполнения BFS с разными реализациями очереди:\n";

    for (int size : graphSizes) {
        std::cout << "\nГраф порядка " << size << ":\n";

        Graph graph(size);
        graph.generateRandomGraph(0.1); // Плотность 10%

        // Измерение времени для стандартной очереди
        double stdQueueTime = measureTime([&]() {
            auto result = graph.bfsWithStdQueue(startVertex);
        });

        // Измерение времени для собственной очереди
        double customQueueTime = measureTime([&]() {
            auto result = graph.bfsWithCustomQueue(startVertex);
        });
    }
}

```

```

});

std::cout << "Стандартная очередь: " << stdQueueTime << " секунд\n";
std::cout << "Собственная очередь: " << customQueueTime << " секунд\n";
std::cout << "Отношение (собственная/стандартная): " << customQueueTime / stdQueueTime << "\n";

// Проверка корректности (должны давать одинаковый результат)
auto result1 = graph.bfsWithStdQueue(startVertex);
auto result2 = graph.bfsWithCustomQueue(startVertex);

if (result1 == result2) {
    std::cout << "Результаты совпадают\n";
}
else {
    std::cout << "Результаты не совпадают!\n";
}
}

// Демонстрация работы на маленьком графе
std::cout << "\n\nДемонстрация на маленьком графе:\n";
std::cout << "=====\n";

Graph demoGraph(6);
demoGraph.addEdge(0, 1);
demoGraph.addEdge(0, 2);
demoGraph.addEdge(1, 3);
demoGraph.addEdge(1, 4);
demoGraph.addEdge(2, 5);

demoGraph.printAdjacencyMatrix();

auto resultStd = demoGraph.bfsWithStdQueue(0);
auto resultCustom = demoGraph.bfsWithCustomQueue(0);

std::cout << "\nBFS с стандартной очередью: ";
for (int vertex : resultStd) {
    std::cout << vertex << " ";
}
std::cout << "\n";

std::cout << "BFS с собственной очередью: ";
for (int vertex : resultCustom) {
    std::cout << vertex << " ";
}
std::cout << "\n";

return 0;
}

```

Задание 1 переделанное под доп задание

```

#include <iostream>
#include <locale.h>
#include <time.h>
#include <cstdio>
#include <queue>
#include <vector>
#include <list>
using namespace std;

```

```

bool is_vert(vector<list<int>>& adjlist, int vertex) {
    return adjlist[vertex].begin() == adjlist[vertex].end();
}

// Обход в ширину для матрицы смежности
void BFS_matrix(int** G, int* vis, int numG, int s) {
    queue<int> q;

```



```

vis[s] = 1;
q.push(s);
while (!q.empty()) {
    int v = q.front();
    q.pop();
    printf("%3d", v);
    for (int i = 0; i < numG; i++) {
        if (vis[i] == 0 && G[v][i] == 1) {
            q.push(i);
            vis[i] = 1;
        }
    }
}
}
}

```

// Обход в ширину для списков смежности

```

void BFS_adjacency_list(vector<list<int>>& adjList, int* vis, int numG, int s) {
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);
        // Проходим по всем соседям вершины v
        for (auto it = adjList[v].begin(); it != adjList[v].end(); ++it) {
            int neighbor = *it;
            if (vis[neighbor] == 0) {
                q.push(neighbor);
                vis[neighbor] = 1;
            }
        }
    }
}
}

```

// Функция для создания списков смежности из матрицы смежности

```

vector<list<int>> create_adjacency_list(int** G, int numG) {
    vector<list<int>> adjList(numG);
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            if (G[i][j] == 1) {
                adjList[i].push_back(j);
            }
        }
    }
    return adjList;
}

```

// Функция для вывода списков смежности

```

void print_adjacency_list(vector<list<int>>& adjList, int numG) {
    cout << "Списки смежности:" << endl;

    for (int i = 0; i < numG; i++) {
        cout << i << ": ";
        auto it = adjList[i].begin();

        if (it == adjList[i].end()) {
            cout << "isol" << " ";
        }
        else { cout << *it << " "; }

        cout << endl;
    }
}

```

```

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int** G;
    int* vis;
    int numG, s;

    cout << "Введите количество вершин графа: ";
    cin >> numG;

    // Выделение памяти для матрицы смежности
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    // Выделение памяти для массива посещенных вершин
    vis = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        for (int j = i; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            }
            else {
                G[i][j] = G[j][i] = rand() % 2; // Симметричная матрица
            }
        }
    }

    cout << "Матрица смежности:" << endl;
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d ", G[i][j]);
        }
        printf("\n");
    }

    cout << "Введите начальную вершину для обхода в ширину: ";
    cin >> s;

    // Инициализация массива посещенных вершин
    for (int i = 0; i < numG; i++) {
        vis[i] = 0;
    }

    cout << "Порядок обхода (матрица смежности): ";
    BFS_matrix(G, vis, numG, s);
    cout << endl;

    vector<list<int>> adjList = create_adjacency_list(G, numG);
    if (is_vert) {
        cout << "выбрана не правильная вершина для обхода";
        return 0;
    }
    print_adjacency_list(adjList, numG);

    for (int i = 0; i < numG; i++) {
        vis[i] = 0;
    }
}

```

```

cout << "Порядок обхода (списки смежности): ";
BFS_adjacency_list(adjList, vis, numG, s);
cout << endl;

// Освобождение памяти
for (int i = 0; i < numG; i++) {
    free(G[i]);
}
free(G);
free(vis);

return 0;
}

```

Псевдокод

1-е задание

НАЧАЛО

Установить русскую локаль
 Инициализировать генератор случайных чисел

ВВЕСТИ numG // количество вершин графа

// Выделение памяти для матрицы смежности и массива посещений
 G = двумерный массив размером [numG][numG]
 vis = массив размером [numG], заполненный нулями

// Генерация матрицы смежности
 ДЛЯ i = 0 ДО numG-1
 ДЛЯ j = 0 ДО numG-1
 ЕСЛИ i == j ТО
 G[i][j] = 0 // нет петель
 ИНАЧЕ
 G[i][j] = случайное(0 или 1)
 G[j][i] = G[i][j] // симметричность
 КОНЕЦ ЕСЛИ
 КОНЕЦ ДЛЯ
 КОНЕЦ ДЛЯ

// Вывод матрицы смежности
 ВЫВЕСТИ "Матрица смежности:"
 ДЛЯ i = 0 ДО numG-1
 ДЛЯ j = 0 ДО numG-1
 ВЫВЕСТИ G[i][j]
 КОНЕЦ ДЛЯ
 ПЕРЕВЕСТИ СТРОКУ
 КОНЕЦ ДЛЯ

ВВЕСТИ s // начальная вершина для обхода

ВЫВЕСТИ "Порядок обхода:"

// Алгоритм BFS
 СОЗДАТЬ очередь q
 vis[s] = 1 // пометить начальную вершину как посещенную

q.добавить(s)

ПОКА q не пуста

v = q.взять_первый() // извлечь вершину из начала очереди
ВЫВЕСТИ v

// Обработка смежных вершин

ДЛЯ i = 0 ДО numG-1

ЕСЛИ vis[i] == 0 И G[v][i] == 1 ТО

vis[i] = 1

q.добавить(i)

КОНЕЦ ЕСЛИ

КОНЕЦ ДЛЯ

КОНЕЦ ПОКА

Освободить память

КОНЕЦ

2-е задание

ПСЕВДОКОД: СРАВНЕНИЕ BFS СО СТАНДАРТНОЙ И СОБСТВЕННОЙ ОЧЕРЕДЬЮ

КЛАСС УЗЕЛ СПИСКА (ListNode):

ПОЛЯ:

data - данные узла

next - указатель на следующий узел

КОНСТРУКТОР(значение):

data = значение

next = null

КЛАСС СОБСТВЕННАЯ ОЧЕРЕДЬ (CustomQueue):

ПОЛЯ:

front - указатель на начало очереди

rear - указатель на конец очереди

queueSize - размер очереди

КОНСТРУКТОР():

front = null

rear = null

queueSize = 0

МЕТОД push(значение):

СОЗДАТЬ новый узел с данным значением

ЕСЛИ rear == null: // Очередь пустая

front = новый узел

rear = новый узел

ИНАЧЕ:

rear.next = новый узел

rear = новый узел

queueSize увеличить на 1

МЕТОД pop():

ЕСЛИ очередь пустая:

ВЫБРОСИТЬ исключение

front = front.next // Сдвинуть начало

ЕСЛИ front == null: // Очередь стала пустой
 rear = null

queueSize уменьшить на 1

МЕТОД getFront() -> значение:
 ЕСЛИ очередь пустая:
 ВЫБРОСИТЬ исключение

 ВЕРНУТЬ front.data

МЕТОД empty() -> логическое:
 ВЕРНУТЬ (front == null)

МЕТОД size() -> число:
 ВЕРНУТЬ queueSize

КЛАСС ГРАФ (Graph):

 ПОЛЯ:

 adjacencyMatrix - матрица смежности

 vertices - количество вершин

 КОНСТРУКТОР(n):

 vertices = n

 СОЗДАТЬ матрицу размером n x n, заполненную нулями

 МЕТОД addEdge(u, v):

 ЕСЛИ u и v в пределах границ:

 adjacencyMatrix[u][v] = 1

 adjacencyMatrix[v][u] = 1 // Неориентированный граф

 МЕТОД generateRandomGraph(плотность=0.3):

 ИНИЦИАЛИЗИРОВАТЬ генератор случайных чисел

 ДЛЯ i ОТ 0 ДО vertices-1:

 ДЛЯ j ОТ i+1 ДО vertices-1:

 ЕСЛИ случайное_число < плотность:

 ВЫЗВАТЬ addEdge(i, j)

 МЕТОД bfsWithStdQueue(startVertex) -> вектор:

 СОЗДАТЬ пустой вектор traversalOrder

 СОЗДАТЬ массив visited размером vertices, заполненный false

 СОЗДАТЬ стандартную очередь q

 visited[startVertex] = true

 q.push(startVertex)

 ПОКА q не пустая:

 current = q.front()

 q.pop()

 ДОБАВИТЬ current в traversalOrder

 ДЛЯ i ОТ 0 ДО vertices-1:

 ЕСЛИ adjacencyMatrix[current][i] == 1 И visited[i] == false:

```
visited[i] = true  
q.push(i)
```

ВЕРНУТЬ traversalOrder

МЕТОД bfsWithCustomQueue(startVertex) -> вектор:
СОЗДАТЬ пустой вектор traversalOrder
СОЗДАТЬ массив visited размером vertices, заполненный false
СОЗДАТЬ собственную очередь q (CustomQueue)

```
visited[startVertex] = true  
q.push(startVertex)
```

ПОКА q не пустая:
current = q.getFront()
q.pop()
ДОБАВИТЬ current в traversalOrder

ДЛЯ i ОТ 0 ДО vertices-1:
ЕСЛИ adjacencyMatrix[current][i] == 1 И visited[i] == false:
visited[i] = true
q.push(i)

ВЕРНУТЬ traversalOrder

МЕТОД printAdjacencyMatrix():
ВЫВЕСТИ "Матрица смежности:"
ДЛЯ i ОТ 0 ДО vertices-1:
ДЛЯ j ОТ 0 ДО vertices-1:
ВЫВЕСТИ adjacencyMatrix[i][j]
ПЕРЕЙТИ НА НОВУЮ СТРОКУ

МЕТОД getVerticesCount() -> число:
ВЕРНУТЬ vertices

ФУНКЦИЯ measureTime(функция) -> число:
start = ТЕКУЩЕЕ_ВРЕМЯ
ВЫЗВАТЬ функцию
end = ТЕКУЩЕЕ_ВРЕМЯ
ВЕРНУТЬ (end - start) в секундах

ОСНОВНАЯ ПРОГРАММА:
УСТАНОВИТЬ локаль

СОЗДАТЬ массив graphSizes = [10, 50, 100, 200, 500, 1000]
startVertex = 0

ВЫВЕСТИ "Сравнение времени выполнения BFS с разными реализациями очереди:"

ДЛЯ КАЖДОГО size ИЗ graphSizes:
ВЫВЕСТИ "\nГраф порядка " + size + ":",

СОЗДАТЬ граф g с size вершинами
ВЫЗВАТЬ g.generateRandomGraph(0.1) // Плотность 10%

// Тестирование стандартной очереди

```

stdQueueTime = measureTime(выполнить g.bfsWithStdQueue(startVertex))

// Тестирование собственной очереди
customQueueTime = measureTime(выполнить g.bfsWithCustomQueue(startVertex))

ВЫВЕСТИ "Стандартная очередь: " + stdQueueTime + " секунд"
ВЫВЕСТИ "Собственная очередь: " + customQueueTime + " секунд"
ВЫВЕСТИ "Отношение (собственная/стандартная): " +
(customQueueTime/stdQueueTime)

// Проверка корректности
result1 = g.bfsWithStdQueue(startVertex)
result2 = g.bfsWithCustomQueue(startVertex)

ЕСЛИ result1 == result2:
    ВЫВЕСТИ "Результаты совпадают"
ИНАЧЕ:
    ВЫВЕСТИ "Результаты не совпадают!"

// Демонстрация на маленьком графе
ВЫВЕСТИ "\n\nДемонстрация на маленьком графе:"
ВЫВЕСТИ "======"

СОЗДАТЬ демо-граф с 6 вершинами
ДОБАВИТЬ ребра: (0,1), (0,2), (1,3), (1,4), (2,5)

ВЫВЕСТИ матрицу смежности демо-графа

resultStd = демо-граф.bfsWithStdQueue(0)
resultCustom = демо-граф.bfsWithCustomQueue(0)

ВЫВЕСТИ "BFS с стандартной очередью: "
ВЫВЕСТИ все элементы resultStd

ВЫВЕСТИ "BFS с собственной очередью: "
ВЫВЕСТИ все элементы resultCustom

ЗАВЕРШИТЬ ПРОГРАММУ

```

Результаты работы программы

1-ое задание

```
Консоль отладки Microsoft Visual Studio
Введите кол-во элементов:
5
 0  1  1  1  0
 1  0  1  1  1
 1  1  0  0  0
 1  1  0  0  1
 0  1  0  1  0
Введите нач вершину: 0
Порядок обхода:  0  1  2  3  4
C:\Users\Admin-PC\Desktop\Lab8\lab8\x64\Debug\lab8.exe (процесс 5868) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

2-е задание

```
Консоль отладки Microsoft Visual Studio
Результаты совпадают

Граф порядка 500:
Стандартная очередь: 0.0058847 секунд
Собственная очередь: 0.0056963 секунд
Отношение (собственная/стандартная): 0.967985
Результаты совпадают

Граф порядка 1000:
Стандартная очередь: 0.022888 секунд
Собственная очередь: 0.023184 секунд
Отношение (собственная/стандартная): 1.01293
Результаты совпадают

Демонстрация на маленьком графе:
=====
Матрица смежности:
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 0 1
0 1 0 0 0 0
0 1 0 0 0 0
0 0 1 0 0 0

BFS с стандартной очередью: 0 1 2 3 4 5
BFS с собственной очередью: 0 1 2 3 4 5

E:\per\8.2\x64\Debug\8.2.exe (процесс 10576) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```


Вывод

В ходе выполнения лабораторной работы были успешно реализованы алгоритмы обхода графа в ширину с использованием различных структур данных. Основной задачей являлось освоение принципов работы с графами и алгоритмом BFS на практике.

Была разработана программа на языке C++, которая генерирует матрицу смежности для неориентированного графа заданного размера. Программа корректно формирует симметричную матрицу, исключая петли и случайным образом устанавливая связи между вершинами. Реализованный алгоритм обхода в ширину демонстрирует правильную работу: начинается с выбранной пользователем вершины, последовательно посещает все достижимые вершины, используя очередь для управления порядком обхода.

Алгоритм BFS показал свою эффективность для обработки связных компонент графа. Использование стандартного класса `queue` из библиотеки STL обеспечило надежную работу с данными и упростило реализацию алгоритма. Программа наглядно отображает матрицу смежности и порядок обхода вершин, что позволяет легко проверить корректность ее работы.

В процессе работы были освоены ключевые аспекты теории графов: представление графов в виде матрицы смежности, принципы обхода в ширину, работа с динамическими структурами данных. Полученные знания могут быть применены для решения более сложных задач на графах, таких как поиск кратчайшего пути или анализ связности.